



FUNDAÇÃO
HERMÍNIO OMETTO

Sistemas de Informação
Sistemas Operacionais I/ 4º A/2025

TRABALHO EM GRUPO - SISTEMA OPERACIONAL

ALGORITMO DE ESCALONAMENTO EM SISTEMAS OPERACIONAIS

Arthur Peixoto Lacerda 116319

Guilherme Henrique Cavarsan 117017

Octavio Thim Dias 117607

Rodolfo Henrique Ribeiro Zanchetta 117179

Prof. Esp. William C. Augustonelli (Billy)



Resumo

Esse trabalho apresenta, de forma prática, o impacto dos algoritmos de escalonamento em três sistemas operacionais: Windows, Linux e Android. Para isso, foi desenvolvido um simulador em Python que comparou os algoritmos FCFS, SJF, Round Robin e CFS em diferentes cenários. Os experimentos geraram tabelas e relatórios interativos, permitindo observar o desempenho de cada abordagem. Os resultados mostraram que o Windows apresentou maior variação entre algoritmos, enquanto o Linux manteve estabilidade. Já o Android mostrou comportamento uniforme, com destaque para a superioridade dos algoritmos preemptivos em termos de responsividade.

Introdução

O escalonamento de processos é um dos elementos fundamentais para o funcionamento eficiente de sistemas operacionais, pois garante a utilização adequada do processador e promove o compartilhamento justo entre múltiplos usuários e aplicações. Diante de múltiplos processos concorrendo por recursos limitados, o algoritmo de escalonamento é responsável por decidir a ordem de execução, impactando diretamente métricas como tempo de resposta, throughput e tempo médio de espera (MACHADO; MAIA, 2019).

Dentre os principais algoritmos de escalonamento estudados na literatura estão o First Come, First Served (FCFS), o Shortest Job First (SJF), o Round Robin (RR) e o Completely Fair Scheduler (CFS). Cada um deles possui características próprias que afetam o desempenho do sistema em diferentes cenários: enquanto o FCFS privilegia a simplicidade, o SJF busca reduzir o tempo médio de espera, o RR favorece sistemas interativos, e o CFS, utilizado no Linux, adota um modelo baseado em fatia de tempo proporcional (MACHADO; MAIA, 2019).

Sendo assim, esse trabalho tem como objetivo comparar o desempenho dos algoritmos de escalonamento nos sistemas Windows, Linux e Android, por meio de simulações experimentais implementadas em Python. A motivação central é colocar em prática os conhecimentos adquiridos em aula e observar, de forma aplicada, como diferentes



sistemas tratam o problema do escalonamento, analisando suas vantagens, limitações e impactos em métricas clássicas de desempenho.

Revisão Teórico/Revisão

Escalonamento de Processos

O escalonamento de processos é a técnica responsável por decidir a ordem de execução das tarefas em um processador, com o objetivo de garantir eficiência no uso dos recursos e justiça entre os processos concorrentes. Essa decisão afeta diretamente métricas como tempo médio de espera, tempo de resposta e throughput (MACHADO; MAIA, 2019).

FCFS (First Come, First Served)

O algoritmo FCFS é baseado na ordem de chegada dos processos. É simples de implementar, mas pode gerar o chamado efeito convoy, no qual processos longos atrasam a execução dos menores. Essa característica limita sua aplicabilidade em sistemas modernos (TANENBAUM, 2015).

SJF (Shortest Job First)

O algoritmo SJF prioriza processos com menor tempo de execução, reduzindo o tempo médio de espera do conjunto. Contudo, pode ocasionar inanição de processos mais longos, caso processos curtos continuem chegando ao sistema (SILBERSCHATZ; GALVIN; GAGNE, 2013).

Round Robin (RR)

O Round Robin é amplamente utilizado em sistemas interativos, pois distribui o tempo do processador de maneira cíclica, com base em uma fatia de tempo (quantum). Essa estratégia garante responsividade, mas seu desempenho depende da escolha adequada do valor do quantum (TANENBAUM, 2015).

CFS (Completely Fair Scheduler)

O CFS é o algoritmo padrão do Linux e adota uma abordagem mais sofisticada, baseada em árvores balanceadas, buscando oferecer justiça e equilíbrio entre os



processos. O algoritmo calcula o tempo virtual de cada processo e tenta igualá-lo ao longo da execução. Essa implementação está documentada oficialmente no Linux Kernel Documentation e no Android Open Source Project (AOSP) (DOCUMENTAÇÃO LINUX, 2023; DOCUMENTAÇÃO AOSP, 2023).

Desenvolvimento

A metodologia adotada neste trabalho baseou-se no desenvolvimento e utilização de um simulador em Python, construído especificamente para avaliar o desempenho de diferentes algoritmos de escalonamento. O simulador foi projetado de forma modular, permitindo a criação e execução de múltiplos cenários de carga de trabalho.

Inicialmente, foram definidos cenários de simulação que contemplaram situações variadas:

- Básico: conjunto reduzido de processos, com tempos de execução heterogêneos.
- Intensivo em CPU: processos longos e de alta demanda, visando avaliar o comportamento em cargas CPU-bound.
- Misto: combinação de processos interativos e batch, simulando ambientes híbridos.
- Personalizados: coleta de métricas reais do ambiente de execução, tanto em desktops quanto em dispositivos móveis.

Para a coleta de dados reais, o simulador integrou-se com:

- Windows 11: utilizando bibliotecas Python para acesso ao monitoramento nativo do sistema (ex.: psutil).
- Linux (WSL2): Execução de comandos nativos como top e ps, permitindo observar o funcionamento sob ambiente Unix-like.

- 
- Android (Samsung Galaxy M14 5G): conexão via ADB (Android Debug Bridge), utilizando comandos como `adb shell top` e `adb shell dumpsys cpuinfo` para obter consumo de CPU e memória em tempo real.

Os algoritmos implementados e testados foram:

- FCFS (First Come, First Served)
- SJF (Shortest Job First) em versões preemptiva e não-preemptiva
- Round Robin (RR) com diferentes valores de quantum
- CFS (Completely Fair Scheduler), conforme documentado no kernel Linux e no Android Open Source Project

Durante a execução dos experimentos, os resultados foram organizados em relatórios interativos em HTML, com tabelas ordenáveis, métricas destacadas e comparações diretas entre os algoritmos. Essa visualização permite identificar rapidamente padrões de desempenho e contrastes entre os sistemas operacionais estudados.

Discussão dos Resultados

A execução dos experimentos nos três ambientes (Windows, Linux e Android) gerou um conjunto de dados que permitiu comparar de forma prática o impacto dos algoritmos de escalonamento.

Windows 11

No ambiente Windows, observou-se uma grande variação entre os algoritmos. O FCFS (First Come, First Served) apresentou desempenho limitado, com tempos médios de resposta significativamente elevados (em torno de 254,25 ms), o que reforça sua ineficiência em cenários modernos com múltiplos processos concorrentes. Por outro lado, o Round Robin ($q=2$) e o CFS se destacaram como as soluções mais responsivas, mantendo tempos de espera e resposta mais baixos. Isso indica na figura 1 que, no Windows, algoritmos preemptivos com divisão justa de CPU são mais adequados para garantir interatividade e boa experiência do usuário.

FIGURA 1 - RELATÓRIO ALGORITMO DE ESCALONAMENTO - WINDOWS 11



Algoritmo	Tempo Médio de Resposta	Tempo Médio de Retorno	Tempo Médio de Espera	Throughput	Posição
FCFS	254.25	291.38	254.25	0.0269	6
SJF (Não-Preemptivo)	5.62	42.75	5.62	0.0269	3
SJF (Preemptivo)	5.62	42.75	5.62	0.0269	4
Round Robin (q=2)	5.12	48.00	10.88	0.0269	1
Round Robin (q=4)	8.38	51.25	14.12	0.0269	5
CFS (Simplified)	5.12	48.50	11.38	0.0269	2

Fonte: Autor Próprio, 2025

Linux (WSL2)

No caso do Linux, os resultados na figura 2 mostraram-se mais homogêneos, com tempos médios próximos entre todos os algoritmos testados. Essa estabilidade pode ser explicada pela maturidade do kernel Linux e pela eficiência do seu modelo de gerenciamento de processos, que tende a reduzir discrepâncias entre algoritmos mesmo sob cargas diferenciadas. Apesar disso, o CFS, nativo do Linux, apresentou desempenho equilibrado e consistente, validando sua adoção como padrão no sistema.

FIGURA 2 - RELATÓRIO ALGORITMO DE ESCALONAMENTO - LINUX WSL2

Relatório Interativo - Algoritmos de Escalonamento

Execução: 8453f0d9 - Gerado em: 2025-09-29 22:57:17
Pasta de origem: c:\users\rodrigo\source\repos\algoritmos-de-escalonamento\algoritmos-de-escalonamento

Sistema Local (Linux 6.6.87.2-microsoft-standard-WSL2)

Algoritmo	Tempo Médio de Resposta	Tempo Médio de Retorno	Tempo Médio de Espera	Throughput	Posição
FCFS	3.50	4.50	3.50	1.0000	1
SJF (Não-Preemptivo)	3.50	4.50	3.50	1.0000	2
SJF (Preemptivo)	3.50	4.50	3.50	1.0000	3
Round Robin (q=2)	3.50	4.50	3.50	1.0000	4
Round Robin (q=4)	3.50	4.50	3.50	1.0000	5
CFS (Simplified)	3.50	4.50	3.50	1.0000	6

Relatório gerado em: 2025-09-29 22:57:17 - Versão: 0.0.1
Este relatório foi gerado automaticamente pelo sistema de escalonamento. Todos os dados são reais e não devem ser usados para fins legais.

Fonte: Autor Próprio, 2025

Android (via ADB)

No Android, os resultados foram ainda mais uniformes, com métricas praticamente iguais entre todos os algoritmos simulados. Isso evidencia que o sistema abstrai a influência direta dos algoritmos clássicos, apresentando uma camada de gerenciamento que homogênea a percepção de desempenho. Em termos práticos, isso significa que o Android prioriza a experiência do usuário por meio de políticas internas de escalonamento do kernel (baseado no CFS), tornando irrelevante a variação entre algoritmos tradicionais durante a coleta de métricas.

FIGURA 3 - RELATÓRIO ALGORITMO DE ESCALONAMENTO - ANDROID (ADB)



Relatório Interativo - Algoritmos de Escalonamento

Execução: 74618588 - Criação em: 2023-09-28 22:34:58
Nota do usuário: 1 - Você pode clicar no documento para ver o documento.

Android (ADB)

Algoritmo	Tempo Médio de Resposta	Tempo Médio de Retorno	Tempo Médio de Espera	Throughput	Posição
FCFS	0.92	2.00	0.92	0.9231	1
SJF (Não-Preemptivo)	0.92	2.00	0.92	0.9231	2
SJF (Preemptivo)	0.92	2.00	0.92	0.9231	3
Round Robin (q=2)	0.92	2.00	0.92	0.9231	4
Round Robin (q=4)	0.92	2.00	0.92	0.9231	5
CFS (Simplified)	0.92	2.00	0.92	0.9231	6

Desenvolvido por: 74618588 - 2023-09-28 22:34:58
Este relatório foi gerado automaticamente pelo sistema de escalonamento de processos.

Fonte: Autor Próprio, 2025

De modo geral, os resultados reforçam que:

- O Windows é mais sensível ao algoritmo utilizado, apresentando variações expressivas.
- O Linux mantém estabilidade, garantindo desempenho consistente em diferentes estratégias.
- O Android abstrai diferenças, priorizando responsividade uniforme independentemente do algoritmo clássico aplicado.

Conclusão

O trabalho comprovou, na prática, que a escolha do algoritmo de escalonamento influencia diretamente o desempenho dos sistemas operacionais, afetando métricas como tempo de resposta, throughput e espera.



No Windows 11, verificou-se que o sistema é altamente sensível ao algoritmo adotado, apresentando grandes variações entre os resultados. Algoritmos simples, como o FCFS, mostraram-se pouco adequados em cenários de alta concorrência, enquanto estratégias preemptivas, como Round Robin e CFS, destacaram-se pela responsividade superior.

Já no Linux (WSL2), observou-se uma maior estabilidade na alocação de CPU, resultado esperado pela robustez do Completely Fair Scheduler (CFS), padrão no kernel. Essa consistência reforça a maturidade do Linux no gerenciamento de processos, mesmo sob cenários diversos.

No Android, os resultados praticamente idênticos entre algoritmos evidenciam a existência de uma camada de abstração que homogeneiza o impacto das estratégias de escalonamento sobre os processos do usuário. Isso demonstra que, no contexto móvel, a prioridade do sistema está na experiência uniforme e responsiva, independentemente da técnica clássica aplicada.

Assim, os objetivos foram alcançados e, como continuidade, sugere-se explorar algoritmos de tempo real e aplicações em sistemas embarcados e críticos ou até mesmo aplicar esses mesmos testes em ambientes de macOS.

Referências Bibliográficas

MACHADO, F. B.; MAIA, L. F. *Arquitetura de Sistemas Operacionais*. 5. ed. Rio de Janeiro: LTC, 2019.

TANENBAUM, A. S. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson, 2015.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2013.

LINUX FOUNDATION. *Linux Kernel Documentation*. Disponível em:

<https://www.kernel.org/doc/>. Acesso em: 29 set. 2025.

ANDROID OPEN SOURCE PROJECT (AOSP). *Android Developers*. Disponível em:

<https://source.android.com/>. Acesso em: 29 set. 2025.